

RaViLa 개발 경과 보고서

radar + vision + text foundation model · 11개 태스크 통합 instruction 모델
2026-07-31 ~ 2026-08-01 · 8B / 32B Qwen3-VL · 5 × NVIDIA B200

이 문서의 모든 수치는 디스크의 산출물(eval.json, probe 출력)에서 읽어 생성했습니다. 기억이나 요약이 아니라 측정값입니다.

요약

목표는 비디오·레이더·ego를 고정 입력으로 두고 instruction만 바꿔 11개 태스크를 모두 텍스트로 출력하는 단일 모델이었습니다. 태스크 확장 자체는 완료했습니다 (태스크 10·11 → 11개 전부). 그 과정에서 구조적 버그 4건을 발견해 고쳤고, 레이더 인코더를 재설계했습니다.

핵심 문제는 모델이 레이더를 거의 쓰지 않는다는 것이었습니다. 진단 결과 원인이 특정됐습니다: 레이더 정보는 언어모델의 은닉 상태까지 온전히 도달하지만(R^2 0.65), 그것을 숫자 토큰으로 내보내는 단계에서만 유실됩니다(상관 0.14). 이 진단에 근거해 손실 함수를 고치는 개입을 시도했으나, 통제군 5개를 갖춘 최종 검정에서 효과가 확인되지 않았습니다($p=0.152$).

보고서의 목적은 성공 사례 제시가 아니라 무엇을 근거로 무엇을 했고 어디서 틀렸는지를 재현 가능하게 남기는 것입니다. 제가 도중에 내린 잘못된 결론 5건도 §7에 명시합니다.

1. 용어 정의

이 보고서에서 반복되는 지표들입니다. 각 항목은 무엇을 재는가와 왜 그것이 필요한가를 함께 적었습니다.

1.1 RCS (Radar Cross Section, 레이더 단면적)

물체가 레이더 전파를 되돌려 보내는 세기를 넓이 단위로 환산한 값이며 단위는 **dBsm**(decibel relative to one square metre)입니다. 금속 트럭은 크고 보행자는 작습니다. 재질·형상·각도에 따라 결정되는 **전자기적 물성**이라 **카메라로는 원리적으로 측정할 수 없습니다**.

이 보고서에서 RCS가 중심 지표인 이유가 바로 그것입니다. '레이더 검출 개수' 같은 양은 장면 혼잡도와 상관되어 카메라만으로도 어느 정도 맞힐 수 있지만, RCS는 그렇지 않습니다. 따라서 **모델이 레이더를 정말 읽는지를 판별하는 데 가장 깨끗한 대상**입니다.

1.2 R^2 (결정계수, coefficient of determination)

어떤 표현(벡터)에서 목표값을 얼마나 복원할 수 있는지를 0~1로 나타냅니다. **1.0이면 완전 복원, 0이면 평균을 찍는 것과 같음, 음수면 평균보다 못함**입니다. 정의는 $R^2 = 1 - (\text{잔차 제곱합})/(\text{전체 제곱합})$ 입니다.

중요한 점은 held-out(학습에 쓰지 않은) 데이터에서 계산해야 한다는 것입니다. 학습 데이터에서 재면 파라미터가 많을수록 무조건 1에 가까워집니다.

1.3 probe R^2 (선형 프로브)

신경망 내부의 어떤 표현에 특정 정보가 들어 있는지 확인하는 표준 기법입니다. 그 표현을 입력으로 하는 **선형 회귀 하나만** 학습시켜 목표값을 예측하고, held-out R^2 를 봅니다. 선형 모델만 쓰는 이유는, 복잡한 모델을 쓰면 '표현에 정보가 있다'가 아니라 '프로브가 똑똑하다'를 재게 되기 때문입니다.

본 보고서에서는 인코더 출력, 커넥터 출력, 언어모델 은닉 상태 세 지점에 각각 프로브를 걸어 **정보가 어느 구간에서 사라지는지**를 특정했습니다(\$6).

주의: 차원이 표본 수보다 훨씬 크면(여기서는 92,160차원 대 수백 행) 정규화 강도에 따라 결과가 완전히 뒤집힙니다. 실제로 이 프로젝트에서 고정 penalty로 갠 값이 R^2 0.5로 나왔다가, penalty를 검증셋에서 고르자 +0.67이 됐습니다(\$7 오류 5).

1.4 shuffled ablation (레이더 교체 통제)

모델에 **다른 클립의 레이더**를 넣고 같은 질문을 시키는 통제입니다. 입력의 통계적 성질은 그대로이고 **대응 관계만** 깨집니다.

zeroed(레이더를 0으로)와 대비됩니다. zeroed는 '레이더가 중요하다'와 '모델이 처음 보는 입력을 만났다'를 구분하지 못합니다. 반면 shuffled에서 성능이 떨어진다면, 모델이 레이더를 **이 장면에 대해** 쓰고 있었다는 뜻입니다.

본 보고서의 핵심 지표인 '레이더 기여분(radar part)'은 full 상관 shuffled 상관으로 정의합니다.

1.5 digit-distance loss (자릿수 거리 손실)

언어모델은 숫자를 자릿수 토큰으로 하나씩 생성합니다. 예를 들어 638은 '6' → '3' → '8' 세 번의 독립적인 15만 분류 문제입니다. 표준 cross-entropy는 **639와 100을 거의 같게 처벌합니다** — argmax가 정답인지만 보기 때문입니다.

digit-distance는 이를 고치려는 시도입니다. 숫자 위치에서 10개 자릿수 토큰으로 분포를 제한하고 **기댓값**을 구해 실제 자릿값과의 거리를 벌점으로 줍니다. gradient가 '8의 확률을 올려라'가 아니라 **'8 쪽으로 이동하라'**가 됩니다.

1.6 Welch's t-test (웰치 t 검정)

두 집단의 평균 차이가 우연으로 설명되는지 판정합니다. 일반 t 검정과 달리 두 집단의 **분산이 다를 수 있다고 가정**하므로, 시드마다 편차가 큰 학습 실험에 적합합니다.

p 값은 '효과가 없는데 이 정도 차이가 우연히 나올 확률'입니다. 관례적으로 0.05 미만이면 효과를 인정합니다. 본 보고서의 최종 검정은 **p = 0.152**로, 효과를 주장할 수 없는 값입니다.

Cohen's d는 차이의 크기를 표준편차 단위로 나타냅니다. d=1.0은 큰 차이지만, 표본이 작으면 d가 커도 p는 유의하지 않을 수 있습니다 — 본 보고서가 정확히 그 경우입니다(d=1.00, p=0.152, n=5+5).

2. 출발점과 목표

데이터셋에는 11개 태스크가 정의되어 있었습니다. 그러나 학습 코드가 실제로 읽는 태스크는 2개(10번 QA, 11번 scene description)뿐이었고, 01~09번을 읽는 브랜치 자체가 없었습니다. 'foundation model'이라는 이름이 실제 구현을 앞서 있었습니다.

사용자의 목표는 명확했습니다: 비디오·레이더·ego는 고정 입력으로 두고, instruction 문장만 바꿔서 11개 태스크를 모두 텍스트로 출력하는 단일 모델. 태스크별 출력 헤드 없이, 전부 언어모델의 생성으로.

2.1 입력 구조 (실측)

샘플 하나를 실제로 토큰화해 측정한 결과입니다. 총 1,437 토큰:

구성	토큰 수	비율	비고
비디오	840	58.5%	20프레임 → 패치 3,360 → 2×2 병합
레이더	240	16.7%	20프레임 × (query 10 + sum 1 + max 1)
텍스트	357	24.8%	system + ego + 질문 + 답
↳ 손실 적용	25	1.7%	assistant 답변만

레이더는 전체의 16.7%이고 비디오는 58.5%입니다. 같은 장면을 두 센서가 보는데 비디오 쪽이 3.5배 많은 토큰으로 표현되어 있습니다. 그리고 손실이 걸리는 것은 전체의 1.7%인 25토큰뿐입니다. 이 구조적 비대칭이 이후 모든 문제의 배경입니다.

2.2 레이더가 모델에 들어가는 경로

레이더 점군 (20, 1024, 8ch)
→ Fourier 특징 + Linear → 384차원
→ spatial self-attention ×4 (프레임 내 1,024점)
→ frame_pool: 학습 query 10개 + sum 토큰 + max 토큰
→ temporal self-attention ×3 → (240, 384)
→ RadarConnector: Linear(384→4096) → (240, 4096)
→ 임베딩 forward hook으로 <|radar_pad|> 자리에 덮어쓰기

inputs_embeds로 넘기지 않고 hook을 쓰는 이유: Qwen3-VL은 input_ids로 비디오 자리를 찾아 비전 타워 출력을 넣는데, inputs_embeds를 주면 둘 중 하나만 받으므로 비전 경로가 통째로 꺼집니다.

3. 발견한 구조적 버그 4건

모두 실제 버그였고, 모두 결과에 영향을 주고 있었습니다.

3.1 레이더 인코더의 출력 경로가 사전학습된 적이 없음

근거: 한 번 backward한 뒤 `p.grad is None`인 파라미터를 세어 봤더니 72개였습니다. temporal 3개 층, global_pool, global_query, frame_pos, norm_out 전부입니다.

원인: 사전학습 손실이 `per_point(moving, box_class)`와 `frame_tokens(ego)`만 사용하고, 언어모델에 실제로 전달되는 tokens 출력에는 어떤 손실도 닿지 않았습니다. 즉 인코더는 랜덤 초기화된 temporal 믹서와 랜덤 투영을 출하하고 있었습니다.

보강 증거: 체크포인트의 `norm_out.weight`가 정확히 전부 1.0이었습니다. 학습된 LayerNorm이 모든 원소를 정확히 1.0으로 유지할 확률은 없습니다.

수정: 방출 토큰에 프레임별 레이더 통계(검출 수, 이동 수, RCS 최대/평균, 최대 거리) 회귀 손실을 추가. 전부 입력에서 계산되므로 라벨 비용 0. gradient 미수신 파라미터가 72개 → 14개로 감소했고, 남은 14개는 self-attention에서 원래 쓰이지 않는 `norm_kv`입니다.

3.2 인코더가 DDP로 감싸이지 않아 rank 간 발산

근거: `configure_stage`가 joint 단계에서 인코더 상단을 학습 대상으로 푸는데, `main()`에서는 `connector`만 `DistributedDataParallel`로 감쌌습니다.

결과: 5개 rank가 각자의 gradient로 따로 스텝을 밟아 가중치가 발산하고, 저장은 rank 0 것만 됐습니다. 실질적으로 1/5 데이터로 학습된 인코더입니다.

3.3 --resume이 문서에만 있고 구현되지 않음

근거: 모듈 docstring에 `--resume checkpoints/vlm_8B_align` 예시가 있는데 `argparse`에도 코드에도 없었습니다.

결과: joint와 full 단계가 커넥터를 매번 랜덤 초기화로 시작했습니다. align이 학습한 커넥터를 버린 것입니다.

영향의 크기: 수정 후 step 1 loss가 2.587 → 0.888로 떨어졌습니다. 커넥터가 실제로 큰 일을 하고 있었다는 직접 증거입니다.

3.4 MoE 체크포인트가 expert 수를 복원하지 않음

체크포인트의 args에서 readout과 frame_queries만 읽고 experts는 빠뜨려, 라우팅 인코더를 dense로 만들려다 로드 실패했습니다. `load_encoder_state`를 엄격하게 만들어 둔 덕에 조용히 틀리지 않고 에러가 났습니다.

4. 레이더 인코더 재설계

근거: 기존 global_pool은 256개 학습 query가 20프레임 전체를 한 번에 압축합니다. 그런데 모든 레이더 질문은 "at frame 11"처럼 프레임을 지목합니다. query가 프레임에 묶여 있지 않으므로 모델이 지목할 대상이 구조적으로 없었습니다.

변경: 프레임 정렬 readout. 프레임당 resample 토큰 10개 + **sum 토큰**(masked sum + log(1+n)) + **max 토큰**(masked max) = $20 \times 12 = 240$ 토큰. 토큰 인덱스 $f \times 12 + k$ 가 프레임 f 에 **구조적으로** 대응합니다.

sum과 max를 별도 토큰으로 둔 이유: attention의 softmax 가중치는 합이 1이므로 출력이 평균이고 점 개수에 거의 불변합니다. 개수를 세려면 sum이, 최댓값을 알려면 max가 필요한데 둘 다 attention이 만들 수 없습니다.

4.1 프로브로 측정한 효과

측정 대상	기존 (global)	신규 (frame)	다중 프로파일
lrr1_n_points	0.314	0.650	0.706
lrr1_n_moving	0.258	0.658	0.626
lrr1_max_rcs	0.286	0.761	0.875
box acc (사전학습)	34.3%	34.3%	43.5%

max_rcs가 0.286에서 0.875로 뒤집힌 것이 max 토큰의 직접적 효과입니다. 대조군인 sumpool은 0.085에 그칩니다 — 합산으로는 극값을 표현할 수 없다는 예측대로이고, 모델이 대조군보다 10배 잘합니다.

'다중 프로파일'은 사전학습을 lrr1 클립(35,948개)에서 lrr1+srr0(78,565개)로 넓힌 것입니다. instruction 모델은 --all-profiles로 클립의 51%에서 SRR을 먹는데, 인코더는 한 번도 SRR을 본 적이 없었습니다.

4.2 센서 조건부 MoE — 실패

SRR(205점)/MRR(338점)/LRR(638점)의 점밀도가 3배씩 다르므로 센서별 expert가 타당해 보였습니다. 구현 후 측정 결과 **dense보다 나빴습니다** (0.648/0.596/0.716 대 0.706/0.626/0.875). 파라미터를 14.7M → 28.9M으로 두 배 늘렸는데 세 지표 모두 하락했습니다. 이후 실험에서 제외했습니다.

5. 핵심 문제: 모델이 레이더를 쓰지 않음

지표: shuffled ablation(다른 클립의 레이더로 교체)했을 때 손실이 얼마나 나빠지는가. 0이면 레이더가 장식입니다.

학습 조건	mean full	레이더 갭	카메라 갭	선택성
구 인코더 + align	1.0742	+0.0459	+0.0433	1.06
구 인코더 + LoRA joint	0.6453	+0.0029	+0.0010	3.00
신 인코더 + align	0.9400	+0.0994	+0.0255	3.89
신 인코더 + full FT	0.6103	+0.0007	-0.0001	--

읽는 법: align 단계(언어모델 동결, 커넥터만 학습)는 레이더 갭이 +0.099로 살아 있고 선택성 3.89 — 레이더가 필요한 태스크에 의존이 집중되어 있습니다. 그런데 **언어모델을 학습시키는 순간 갭이 +0.0007로 붕괴합니다.** LoRA(1.1억 파라미터)든 full FT(88억)든 마찬가지이고, **용량이 클수록 더 심합니다.**

이 시점의 해석은 '언어모델을 학습시키면 레이더를 버린다'였습니다. §6의 진단이 이 해석을 정정하게 됩니다.

5.1 사전분포 기준선 — 얼마나 못 쓰는가

절대 수치를 해석하려면 기준선이 필요합니다. 레이더를 전혀 보지 않고 자리별 최빈 숫자만 찍는 모델의 digit 정확도를 데이터에서 직접 계산했습니다:

radar_probe 질문	답 형식	사전분포만
strongest radar return (dBsm)	2자리	24.0%
illuminated / camera-only	2자리	32.7%
detections / moving	5자리	12.3%
가중 평균		23.9%

당시 최고 모델이 27.5%였습니다. 기준선 대비 +3.6포인트 — **사용 가능한 정보의 약 5%만** 쓰고 있었습니다.

6. 시도한 개입 7건

각각 무엇을 근거로 시도했고 왜 실패했는지 적습니다.

개입	건드린 대상	근거	결과
radar dropout 0.25	데이터	레이더 없이 못 풀게 강제	실패
contrast hinge	손실(분포)	shuffled 갭을 직접 최적화	△ 갭만 부풀림
CoT rationale	데이터	레이더 수치를 먼저 말하게	실패 (환각)
답 구간화	출력 형식	정밀도가 병목이라 가정	실패
레이더 가중치 6배	mixture	노출 부족이라 가정	실패
센서 MoE	표현	센서별 분포 차이	실패 (dense보다 나쁨)
digit-distance 손실	손실(수치)	\$7 진단에 근거	미확인 (p=0.152)

6.1 contrast hinge — 지표만 오르는 사례

다른 클립의 레이더가 최소 0.15 nats 더 나빠야 한다는 hinge를 손실에 추가했습니다. 결과는 극적으로 보였습니다: 레이더 갭 +0.2325(align의 2.3배).

그러나 hinge가 평가 지표를 직접 최적화합니다 — 학습의 roll(1)과 평가의 shuffled가 같은 연산입니다. 독립 지표인 digit 정확도로 보면 26.9%로 대조군 26.2%와 차이가 없었습니다. 출력 분포를 레이더에 민감하게 만들었을 뿐 숫자를 맞히는 능력은 오르지 않았습니다.

6.2 CoT — 자기 일관적 환각

레이더 수치가 정답보다 상류에 있는 태스크에만 적용했습니다. 특히 agent_traj: 시선속도(Doppler)가 미래 거리를 물리적으로 결정하므로, rationale이 속도를 말해야 답이 나옵니다.

rationale의 사실성은 검증했습니다: 'closing'이라고 말한 625건 중 92.2%에서 실제로 거리가 줄었고, 'receding' 433건 중 79.9%, 'holding' 238건 중 99.2%였습니다 (무조건 closing 추측은 51.8%).

그런데 학습 결과: loss는 1.83 → 0.25로 급락했지만 레이더를 다른 클립 것으로 바꿔도 digit 정확도가 전혀 떨어지지 않았습니다(+0.0p). 모델은 rationale 형식을 유창하게 배웠지만 거기 쓰는 숫자를 레이더에서 읽는 게 아니라 지어냅니다. 근거 있어 보이는 출력이 실제로는 근거가 없습니다.

7. 진단: 정보가 어디서 사라지는가

근거: 여섯 개입이 모두 실패했다는 것은 문제를 잘못 짚고 있다는 신호입니다. 그래서 '어떻게 고칠까'를 멈추고 '어디서 사라지는가'를 재기로 했습니다.

방법: RCS(카메라로 측정 불가능한 양)를 목표로, 파이프라인 네 지점에 각각 선형 프로브를 걸었습니다. 같은 항목, 같은 프로브, 같은 정규화.

체크포인트	encoder	connector	hidden	hidden (레이더 교체)
vlm_8B_align_fixed	0.670	0.685	0.654	0.008
vlm_8B_full_resume	0.673	0.686	0.650	0.003
vlm_8B_sw_cmp_base	0.672	0.687	0.656	0.009

결과 해석

- ① 인코더(0.67) → 커넥터(0.69) → **언어모델 은닉 상태(0.65)**까지 정보가 거의 손실 없이 도달합니다. 답의 첫 토큰을 생성하는 바로 그 벡터에서 RCS를 R^2 0.65로 선형 복원할 수 있습니다.
- ② 다른 클립의 레이더를 넣으면 **0.65 → 0.005로 완전 붕괴**합니다. 즉 은닉 상태의 RCS는 전적으로 레이더에서 온 것이고 카메라 기여는 0입니다.
- ③ 그런데 모델이 실제로 쓰는 숫자는 상관 0.14 — R^2 로 환산하면 **0.02**입니다. **가진 정보의 3%만 출력**합니다.
- ④ 세 체크포인트(aligned 0.654 / full FT 0.650 / 12k full 0.656)가 **사실상 동일**합니다.

레이더 → 인코더 0.67 → 커넥터 0.69 → 은닉상태 0.65 → 생성 숫자 0.02
(교체 시 0.005) ↑
레이더 기원 확인 여기서만 무너짐

이 진단이 정정한 것

§5에서 '언어모델을 학습시킬수록 레이더를 버린다'고 해석했습니다. **부정확했습니다**. 표현은 전혀 나빠지지 않습니다 — 언제나 R^2 0.65입니다. 달라지는 것은 **출력이 그 표현을 참조하는 정도**뿐입니다.

그리고 앞선 여섯 개입이 실패한 이유가 설명됩니다: **전부 표현이나 데이터를 건드렸는데, 표현은 처음부터 멀쩡했습니다**. 손대지 않은 유일한 것이 은닉 상태를 토큰으로 바꾸는 손실 함수였습니다.

8. digit-distance 손실과 최종 검정

근거: §7의 진단이 지목한 유일한 미개입 지점. cross-entropy는 638에 대해 639와 100을 거의 같게 처벌하므로, 은닉 상태의 연속량을 자릿수로 옮기는 데 부적합합니다.

8.1 1차 결과 — 그리고 성급한 결론

radar_probe 단독 학습에서 시드 2개로 레이더 기여분이 **0.368 / 0.292**가 나왔고, 대조군은 **0.070**이었습니다. 저는 이를 **'4.7배, 진단이 검증됐다'**고 보고했습니다.

문제: 대조군이 한 개였습니다. 처리군 자체가 0.22~0.58로 흩어져 있는데 통제가 하나면 효과 크기를 말할 수 없습니다.

8.2 통제군을 채운 최종 검정

seed	digit w=1.0	control w=0
0	+0.369	+0.070
1	+0.292	+0.218
2	+0.583	+0.271
3	+0.220	+0.176
4	+0.419	+0.451

	n	평균	표준편차	범위
처리군 (w=1.0)	5	+0.377	0.138	+0.220 .. +0.583
대조군 (w=0)	5	+0.237	0.141	+0.070 .. +0.451

차이 +0.139 · Welch t = 1.58 · p = 0.152 · Cohen d = 1.00 · 범위 중첩

판정: 효과를 확인할 수 없습니다. 대조군 seed 4가 0.451로 처리군 4개보다 높습니다. 1차 보고의 '4.7배'는 대조군을 5개 중 최저값(0.070) 하나로 잡은 결과였습니다. 대조군 평균은 0.237입니다.

8.3 측정 자체의 한계

레이더 기여분의 시드 편차가 처리·대조 양쪽에서 **0.14**로, 관측된 차이(0.139)와 같은 크기입니다. 이 예산(12k 샘플, 평가 RCS 항목 약 200개)에서는 이 지표로 어떤 개입도 검증할 수 없습니다. 개입을 더 시도하기 전에 측정을 안정화해야 합니다.

9. 제가 내린 잘못된 결론 5건

모두 통제가 부족한 상태에서 결론을 낸 것이 공통 원인입니다. 재현하는 분들이 같은 함정을 피하도록 남깁니다.

#	주장	무엇이 틀렸나	정정
1	contrast hinge는 Goodhart	resume와 결합 시 digit gap도 상승	부분 정정
2	full FT가 catastrophic forgetting을 일으킬 것	held-out이 오히려 최고(3.007)	예측 실패
3	노출을 늘려도 무용	노출 증가와 태스크 제거를 동시에 변경	설계 결함
4	radar_probe는 카메라로 원리적으로 못 푼다	검출 개수는 장면 혼잡도와 연동	완전히 틀림
5	생성 상관 0.87 달성	질문 형식 3종의 답 크기가 100배 차이 → '어느 질문인가'를 맞힌 것	아티팩트

추가로, 측정 도구 자체에서 잡은 오류 3건:

- **프로브 정규화**: 166행 대 92,160차원에 고정 penalty를 써서 R^2 0.5가 나왔습니다. 같은 양을 더 큰 표본으로 잰 값이 +0.88이었기에 의심할 수 있었고, penalty를 검증셋에서 고르자 +0.67이 됐습니다.
- **배치 1에서의 shuffle**: `roll(shifts=1, dims=0)`은 행이 하나면 자기 자신입니다. full과 shuffled가 소수점까지 동일하게 나와서 발견했습니다. 이전 아이템의 레이더를 쓰도록 고쳤습니다.
- **백그라운드 stdin 상속**: `while read` 루프에서 백그라운드 잡이 stdin을 물려받아 설정 목록을 읽어 먹었습니다. < /dev/null로 격리했습니다.

10. 산출물과 다음 단계

10.1 코드

파일	역할
datatools/frame_objects.py	태스크 01~06 텍스트 타겟 생성 (3.9M 아이템)
training/radar_encoder.py	프레임 정렬 readout, sum/max 토큰, 센서 MoE
training/instruct_data.py	11개 태스크 로더, 센서 프로파일, radar dropout
training/train_vlm.py	align / joint / full(FSDP2) 3단계
training/probe_radar_tokens.py	인코더 출력 프로브
training/probe_pipeline.py	4지점 프로브 — §7의 진단
training/eval_numeric.py	생성 기반 수치 평가 — §8의 판정
training/compare_runs.py	전체 체크포인트 비교표

10.2 로그

runs/ 아래 5개 디렉터리로 정리했습니다: 01_data_prep, 02_encoder, 03_vlm_sweeps, 04_diagnostics, 05_metrics. runs/README.md에 읽는 순서를 적었습니다.

10.3 다음에 해야 할 일

측정을 먼저 고쳐야 합니다. 지금은 개입의 효과보다 시드 노이즈가 큼니다.

- ① **평가 표본 확대** — RCS 항목을 200개에서 2,000개로. 노이즈가 $\sqrt{10}$ 배 줄어듭니다. 이미 학습해 둔 20여 개 체크포인트를 다시 채점하기만 하면 되므로 학습 비용이 없습니다.
- ② **학습 예산 확대** — 12k(1/5 에폭)에서 60k(1 에폭)로. 시드 간 수렴 차이가 줄어듭니다.
- ③ 그 뒤에야 **RLVR(GRPO)**이 의미를 갖습니다. §7의 진단은 정보 흐름이 존재함을 보였으므로 '강화학습이 없는 정보를 만들 수 없다'는 반론은 이 경우 적용되지 않습니다. 다만 **효과를 판별할 수 있는 측정이 먼저입니다.**

보고서 생성: `python -m datatools.build_report`